

채점 서버가 죽으면, 처리 중이던 제출은 누가 책임지나?

제출 원본은 MySQL에 안전하게 남는다. 문제는 채점 서버가 가져간 작업이 그 서버와 함께 주인을 잃는다는 것이다 — 제출은 “채점 중”인 채로 남고, 참가자는 결과를 받지 못한다. 풀어야 할 것은 데이터 유실이 아니라 **진행 중인 작업의 소유권**이었다.

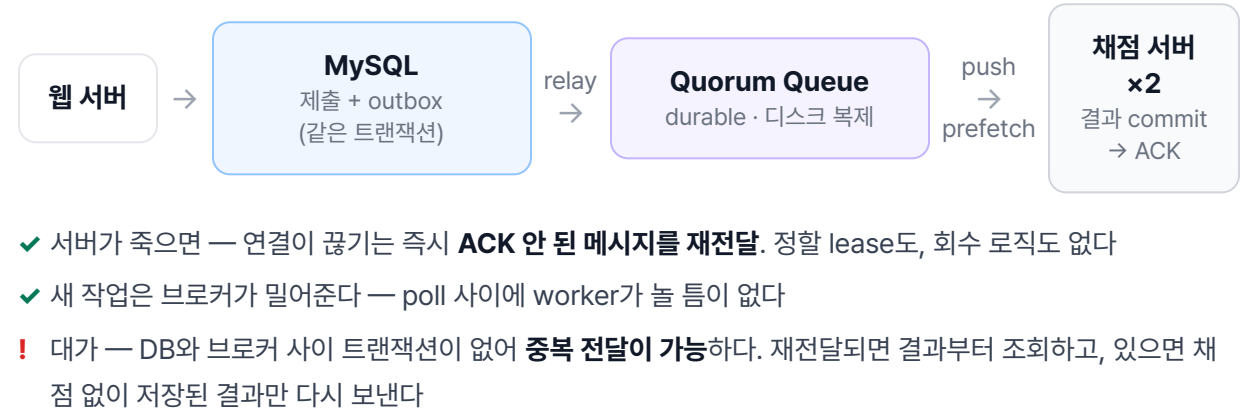
① 첫 구조 — MySQL claim / lease

새 인프라 없이, 제출이 있는 DB에서 해결



② 옮긴 구조 — outbox + RabbitMQ

작업의 주인을 브로커의 ACK 상태로



옮긴 이유 — lease에는 고를 기준이 없는 트레이드오프가 있다

짧게	고를 기준이 없다	길게
오래 걸린 정상 채점을 죽은 것으로 보고 중복 실행		진짜 장애에서 회수가 늦다
소유를 “시간이 지났으니 죽었을 것”이라는 추론이 아니라 “받았고 아직 확인하지 않았다”는 상태로 판단하면, 이 선택 자체가 사라진다고 봤다. 이 판단은 이론이었다.		

그래서 브로커에 기대한 것

측정은 옮긴 뒤, 두 방식을 모두 남겨 두고 했다

① 더 빠른 회복

lease를 기다리지 않고 **즉시 재전달** 되니 장애 때 사용자 지연이 줄어들 것

② 덜 노는 채점 서버

poll 공백이 없으니 **같은 서버로 더 많이** 채점할 것

③ 가벼운 DB

claim·완료 갱신이 사라지니 제출을 받는 **DB의 경합이 줄 것**

재 보니, 트레이드오프는 실재했지만 풀던 문제가 달랐다

기대 ①(더 빠른 회복)부터 확인했다. lease가 정말 고를 수 없는 값인지, 그리고 회복 속도가 사용자 지연을 좌우하는지 — MySQL 방식을 그대로 남겨 두고 두 질문을 따로 켜.

측정 ① — 짧은 lease의 비용은 실재했다 110 RPS · in-flight 16 · worker 16/node · 채점 95% 50ms / 5% 2s · 각 1회 측정 ② — 지연을 좌우한 건 lease가 아니었다

지표	lease 2.5초	lease 1초	읽는 법
실제 중복 채점	0건	354건	전체의 4.9% · 전부 2초짜리 작업
계산상 worker 비용	기준	+67.7%	5%만 중복됐는데 긴 작업이라 비용은 2/3 늘었다
중복 안 된 50ms 작업 p95	715ms	16,358ms	중복이 worker를 점유해 멸절한 작업까지 밀렸다
유실 · 최종 불일치	0 / 0	0 / 0	정합성은 지켜졌다 — 비용과 지연의 문제

채점 서버 1대 SIGKILL · 짧은 작업 p95	lease 4초	lease 10초
50 RPS 남은 1대가 감당	0.88s	0.44s
100 RPS 남은 1대의 용량 초과	10.5s	10.1s

lease가 바꾼 건 죽는 순간 잡혀 있던 수십 건(50 RPS 4건, 100 RPS 20건)의 회수 시점뿐. 전체 지연은 남은 서버가 들어오는 제출을 감당하느냐로 갈렸다 — 감당하면 1초 미만, 못 하면 lease와 무관하게 약 10초.

하지만 안전한 값은 고를 수 있었다. in-flight 64 · 100 RPS에서 lease 2초는 중복 163건을 만들었지만 4초와 10초는 모두 0건, 처리량도 같았다. 돌이켜 보면 heartbeat로 “오래 걸리는 작업”과 “죽은 서버”를 구분할 수도 있었지만, 4초에서 이미 중복이 없어 그 복잡도를 더할 이유가 없었다.

그래서 생각이 바뀐 것

이론의 트레이드오프	실재했다. 하지만 4초로 풀렸고, 사용자 경험을 좌우하는 요인이 아니었다. 이론만 보고 내린 결정이었다.
대회에서의 회복 속도	대회 부하는 평시 위에 몇 분짜리 5배 피크, 그 안에 짧은 10배 피크가 얹힌다. 5배 구간에서 대기열이 쌓이지 않게 용량을 잡으면 평시엔 한 대가 죽어도 여유가 충분하고, 피크엔 참가자가 이미 수십 초를 기다린다. 어느 쪽에서도 회복 속도는 체감되기 어렵다.
남은 기대 ② ③	처리량과 DB는 아직 모른다. 앞서 1,000 starts/s burst를 1회 재 봤지만 진입부 503이 섞여 근거로 쓰지 않았다.

문제는 회복 속도가 아니라 용량이었다.

그렇다면 브로커를 계속 돌 이유는 하나뿐이다 — 같은 피크 경험을 더 적은 자원으로 주는가. 이것을 대회 모양의 부하로 다시 켜.

회복 속도가 아니라 용량이 문제라면, 그 비용은 누가 치르는가

질문을 바꿨다 — 브로커는 **같은 피크 경험을 더 적은 자원으로 주는가?** 평소 위에 5배·10배 피크가 없힌 대회 모양의 부하를 만들고, 두 방식에 같은 채점 용량을 준 뒤 대기열이 얼마나 오래 남는지를 비교했다.

실험 설계

- **변수** — 채점 worker 수로 정한 용량 = 평시의 5.4 / 6.8 / 8.1배
- **지표** — p99 대신 **대기열이 남아 있는 시간**. p99는 대회가 길수록 평시 제출에 희석되어 피크를 가린다
- **규칙** — 반복 실행 · 503이 1건이라도 나오면 버린다(거절된 제출이 통계에서 빠지므로)

세 가지 기대의 최종 결과	결과	근거
장애 때 더 빨리 회복	아니었다	피크 중 채점 서버를 죽여도 두 방식 모두 대기열 170초 . 재전달 지연을 0·4·30초로 바꿔 넣어도 같았다
채점 서버가 덜 논다	조금	실제 용량이 명목 대비 MySQL 77~80% , 브로커 79~84% — 차이 3~5%p
DB 경합이 준다	예상보다 컸다	처리량을 14배로 키우자 드러났다 — 아래

스스로 의심한 것 — MySQL을 덜 튜닝했나(poll 100→20ms: 처리량 그대로, DB CPU만 +34~44%) · 가짜 채점이라 그런가(실제 CPU 연산으로 바뀌도 같은 결론). 어느 쪽도 결론을 바꾸지 않았다.

차이는 DB에서 났다 — 평소 70 RPS(14배) · 같은 용량 배수 · MySQL 컨테이너 CPU 한도 2코어



피크가 끝나도 MySQL 방식은 대기열을 비우는 동안 **120초 중 18~84초** 더 막혔다(브로커 방식 0초). claim과 완료 처리가 **유입이 아니라 채점 속도를 따라** DB를 두드리기 때문이다. 그리고 이 DB는 **유실되면 안 되는 제출 원본도 저장한다** — 이 차이는 비용이 아니라 위험이다.

기대는 채점 서버였지만, 브로커가 아낀 건 DB였다.

장애 지연도 채점 처리량도 크게 달라지지 않았다. 달라진 건 피크에 제출 저장과 같은 DB가 남기는 여유였다. 어느 방식이든 필요한 채점 서버는 명목이 아니라 **실측 용량**(명목의 67~84%, 규모가 클수록 낮았다)으로 계산해야 하고, 이 값 하나로 대기열 길이가 재현됨을 모델로 확인했다.

다시 설계한다면

지금 규모에서는 **MySQL 방식도 충분히 유효하다** — 채점 효율 차이는 3~5%p이고 운영할 인프라가 하나 적다. 브로커를 유지하는 이유는 채점 효율이 아니라 **DB 여유**이고, 처리량이 커져 그 여유가 제출 저장을 위협할 때 의미가 커진다. 그래서 피크 중 DB CPU 여유를 지켜볼 지표로 둔다.